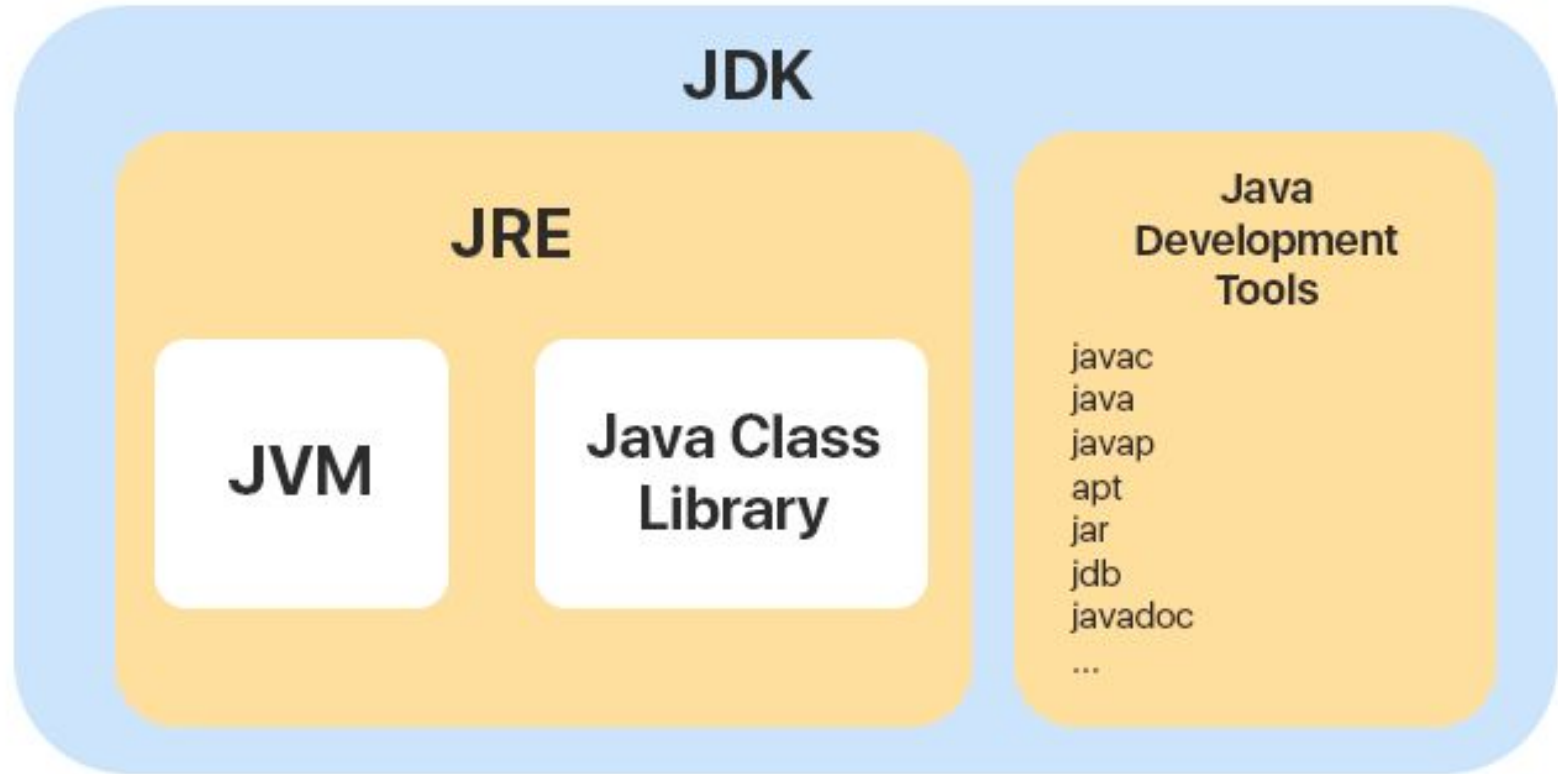

Java

Память в Java.
Garbage Collection

JVM, JRE, JDK



JVM

JVM — это программа, предназначенная для выполнения байт-кода. Первое преимущество JVM — это принцип “Write once, run anywhere”. Он означает, что приложение, написанное на Java, будет работать одинаково на всех платформах. Это является большим преимуществом JVM и самой Java.

Существует множество реализаций JVM, как коммерческих, так и с открытым кодом. Одна из целей создания новых JVM — увеличение производительности для конкретной платформы. Каждая JVM пишется под платформу отдельно, при этом есть возможность написать ее так, чтобы она работала быстрее на конкретной платформе. Самая распространённая реализация JVM — это JVM Hotspot от [OpenJDK](#). Также есть реализации [IBM J9](#), [Excelsior JET](#).

Выполнение кода на JVM

Согласно спецификации Java SE, для того, чтобы получить код, работающий в JVM, необходимо выполнить 3 этапа:

1. Загрузка байт-кода и создание экземпляра класса Class

Грубо говоря, чтобы попасть на JVM, класс должен быть загружен. Для этого существуют отдельные класс-загрузчики..

2. Связывание или линковка

После загрузки класса начинается процесс линковки, на котором байт-код разбирается и проверяется. Процесс линковки в свою очередь происходит в 3 шага:

- verification или проверка байт-кода: проверяется корректность инструкций, возможность переполнения стека на данном участке кода, совместимость типов переменных; проверка происходит один раз для каждого класса;

- preparation или подготовка: на данном этапе в соответствии со спецификацией выделяется память под статические поля и происходит их инициализация;

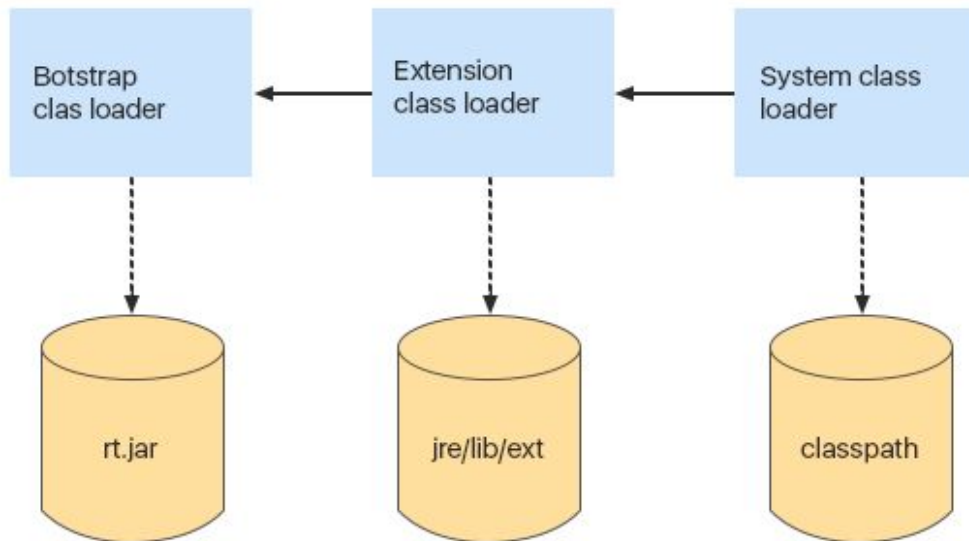
- resolution или разрешение: разрешение символьных ссылок (когда в байт-коде мы открываем файлы с расширением .class, мы видим числовые значения вместо символьных ссылок).

3. Инициализация полученного объекта Class

На последнем этапе класс, который мы создали, инициализируется, и JVM может начинать его исполнение.

Загрузчики классов и их иерархия

Загрузчики классов - это специальные классы, которые являются частью JVM. Они загружают классы в память и делают их доступными для выполнения. Загрузчики работают со всеми классами.



Bootstrap classloader

Первый загрузчик классов — это **Bootstrap classloader**. Он написан на C++. Это базовый загрузчик, который загружает все системные классы из архива `rt.jar`. При этом, есть небольшое отличие между загрузкой классов из `rt.jar` и наших классов: когда JVM загружает классы из `rt.jar`, она не выполняет все этапы проверки, которые выполняются при загрузке любого другого класс-файла т.к. JVM изначально известно, что все эти классы уже проверены. Поэтому, включать в этот архив какие-либо свои файлы не стоит.

Extension classloader и System classloader

Extension classloader загружает классы расширений из папки `jre/lib/ext`. Допустим, вы хотите, чтобы какой-то класс загружался каждый раз при старте Java машины. Для этого вы можете скопировать исходный файл класса в эту папку, и он будет автоматически загружаться.

System classloader загружает классы из `classpath`'а, который мы указали при запуске приложения.

Процесс загрузки классов

В первую очередь мы запрашиваем поиск в кэше **System Class Loader** (кэш системного загрузчика содержит классы, которые уже были им загружены);

Если класс не был найден в кэше системного загрузчика, мы смотрим кэш **Extension class loader**;

Если класс не найден в кэше загрузчика расширений, класс запрашивается у загрузчика **Bootstrap**.

Если класс не найден в кэше **Bootstrap**, он пытается загрузить этот класс. Если **Bootstrap** не смог загрузить класс, он делегирует загрузку класса загрузчику расширений. Если на этот момент класс будет загружен, он остается в кэше у **Extension classloader**, а загрузка класса является завершенной.

Пример

```
import java.sql.DriverManager;  
import java.util.ArrayList;  
public class Program {  
    public static void main(String[] args) {  
        System.out.println("Classloader of this class:" + Program.class.getClassLoader());  
        System.out.println("Classloader of DriverManager:" + DriverManager.class.getClassLoader());  
        System.out.println("Classloader of ArrayList:" + ArrayList.class.getClassLoader());  
    }  
}
```

Classloader of this class:jdk.internal.loader.ClassLoaders\$AppClassLoader@6f94fa3e
Classloader of DriverManager:jdk.internal.loader.ClassLoaders\$PlatformClassLoader@41629346
Classloader of ArrayList:null

Структура Class-файлов и процесс загрузки

Один класс, написанный на Java, компилируется в один файл с расширением .class. Если в нашем Java файле лежит несколько классов, один файл Java может быть скомпилирован в несколько файлов с расширением .class — файлов байт-кода данных классов.

Все числа, строки, указатели на классы, поля и методы хранятся в Constant pool — области памяти Meta space. Описание класса хранится там же и содержит имя, модификаторы, супер-класс, супер-интерфейсы, поля, методы и атрибуты. Атрибуты, в свою очередь, могут содержать любую дополнительную информацию.

Таким образом, при загрузке классов:

- происходит чтение класс-файла, т.е проверка корректности формата
- создается представление класса в Constant pool (Meta space)
- грузятся супер-классы и супер-интерфейсы; если они не будут загружены, то и сам класс не будет загружен

Исполнение байт-кода на JVM

В первую очередь, для исполнения байт-кода, JVM может его интерпретировать. Интерпретация — довольно медленный процесс. В процессе интерпретации, интерпретатор “бежит” построчно по класс-файлу и переводит его в команды, которые понятны JVM.

Также JVM может его транслировать, т.е. скомпилировать в машинный код, который будет исполняться непосредственно на CPU.

Команды, которые исполняются часто, не будут интерпретироваться, а сразу будут транслироваться.

Компиляция

Компилятор — это программа, которая преобразует исходные части программ, написанные на языке программирования высокого уровня, в программу на машинном языке, “понятную” компьютеру.

Компиляторы делятся на:

- **Не оптимизирующие**
- **Простые оптимизирующие** (Hotspot Client): работают быстро, но порождают неоптимальный код
- **Сложные оптимизирующие** (Hotspot Server): производят сложные оптимизирующие преобразования прежде чем сформировать байт-код.

Также компиляторы могут классифицироваться по моменту компиляции:

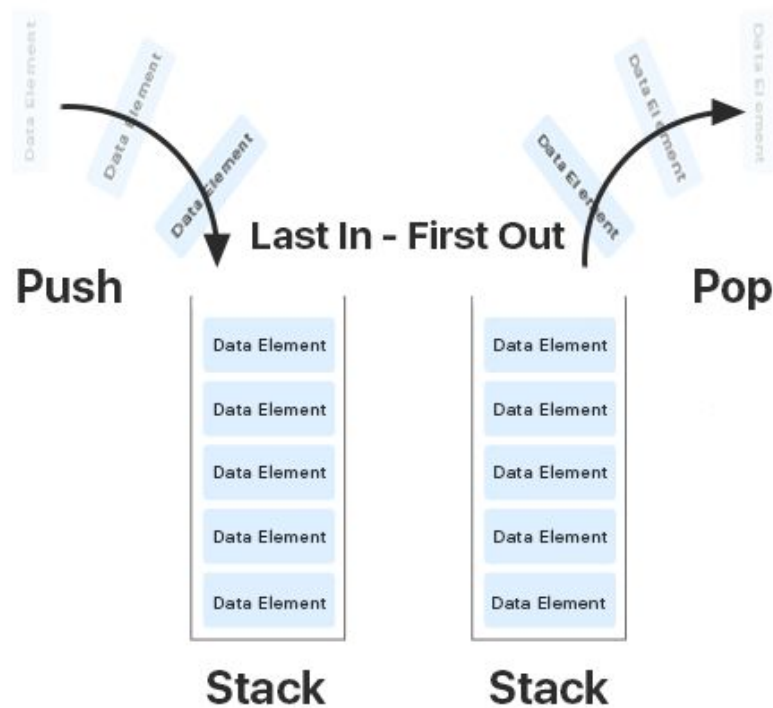
- **Динамические компиляторы:** Работают одновременно с программой, что сказывается на производительности. Важно, чтобы эти компиляторы работали на коде, который часто исполняется. Во время исполнения программы JVM знает, какой код выполняется чаще всего, и, чтобы постоянно не интерпретировать его, виртуальная машина сразу переводит его в команды, которые уже будут исполняться непосредственно на процессоре.
- **Статические компиляторы:** Дольше компилируют, но порождают оптимальный код для исполнения. Из плюсов: не требуют ресурсов во время исполнения программы, каждый метод компилируется с применением оптимизаций.

Организация памяти в Java

Стек (Stack) — это область памяти в Java, которая работает по схеме LIFO — “Last in — First Out” или “Последним вошел, первым вышел”.

Он нужен для того, чтобы хранить методы. Переменные в стеке существуют до тех пор, пока выполняется метод в котором они были созданы.

Если память стека будет заполнена, Java бросит исключение `java.lang.StackOverflowError`.



Пример java.lang.StackOverflowError

```
public class Program {  
    public static void main(String[] args) {  
        Program.recursiveMethod(1);  
    }  
    public static void recursiveMethod(int num) {  
        if (num == 0)  
            return;  
        else  
            recursiveMethod(++num);  
    }  
}
```

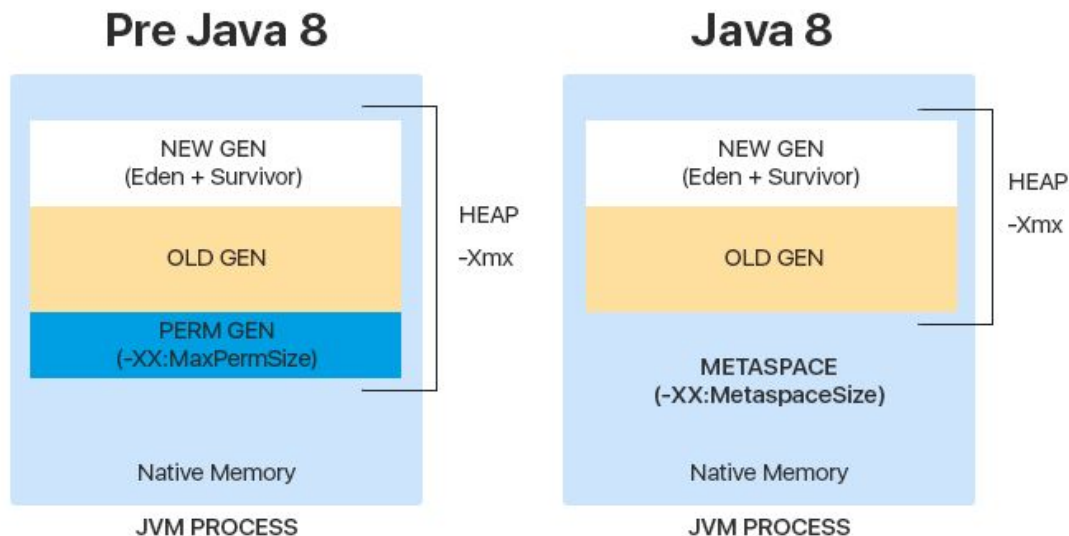
```
Exception in thread "main" java.lang.StackOverflowError  
    at ex10.Program.recursivePrint(Program.java:14)  
    at ex10.Program.recursivePrint(Program.java:14)  
    at ex10.Program.recursivePrint(Program.java:14)  
    at ex10.Program.recursivePrint(Program.java:14)  
    at ex10.Program.recursivePrint(Program.java:14)
```

Ключевые особенности стека

- **Стек** заполняется и освобождается по мере вызова и завершения новых методов
- Доступ к этой области памяти осуществляется быстрее, чем к **куче**
- Размер **стека** определяется операционной системой
- Является потокобезопасным, поскольку для каждого потока создается свой отдельный стек

Heap (куча)

Heap или куча. Она используется для хранения объектов и классов. Новые объекты всегда создаются в куче, а ссылки на них хранятся в стеке. Все объекты в куче имеют глобальный доступ, то есть к ним можно получить доступ из любой точки приложения.



Пример java.lang.OutOfMemoryError

```
public class Program {  
    public static void main(String[] args) {  
        StringBuffer sb = new StringBuffer();  
        while (true) {  
            Program a = new Program();  
            sb.append(a.toString());  
        }  
    }  
}
```

```
Exception in thread "main" java.lang.OutOfMemoryError: Create breakpoint : Java heap space  
    at java.base/java.util.Arrays.copyOf(Arrays.java:3537)  
    at java.base/java.lang.AbstractStringBuilder.ensureCapacityInternal(AbstractStringBuilder.java:228)  
    at java.base/java.lang.AbstractStringBuilder.append(AbstractStringBuilder.java:582)  
    at java.base/java.lang.StringBuffer.append(StringBuffer.java:313)  
    at ex10.Program.main(Program.java:8)
```

Ключевые особенности heap

Когда эта область памяти заполняется полностью, Java бросает `java.lang.OutOfMemoryError`.

Доступ к куче медленнее, чем к стеку.

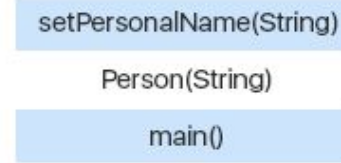
Для сбора неиспользуемых объектов работает сборщик мусора.

Куча, в отличие от стека, не является потокобезопасной, так как любой поток может получить к ней доступ.

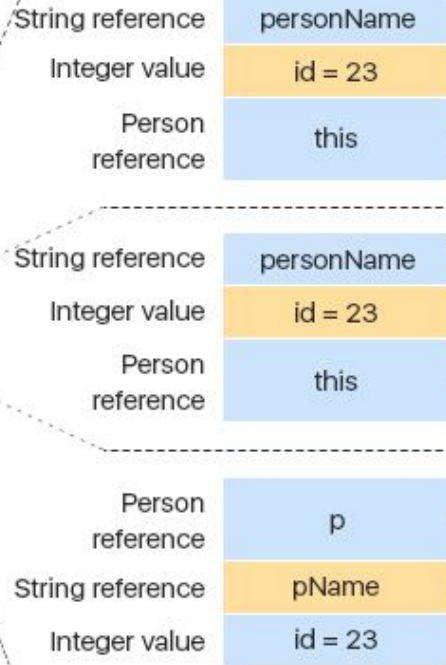
Работа с памятью на примере

```
public class Program {  
    public static void main(String[] args) {  
        int id = 23;                // примитив id > стек  
        String pName = "Jon";      // ссылка > стек, значение > StringPool  
        Person p = null;           // ссылка > стек, значения пока нет.  
        p = new Person(id, pName); // создаётся объект класса Person  
    }  
}  
  
class Person {  
    int personId;  
    String personName;  
    public Person(int personId, String personName) {  
        this.personId = personId;  
        this.setPersonName(personName);  
    }  
    public void setPersonName(String personName) {  
        this.personName = personName;  
    }  
}
```

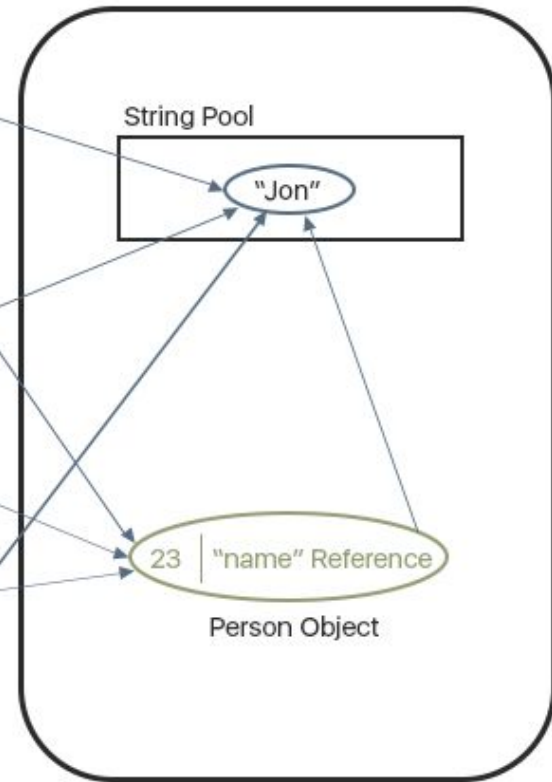
Call Stack



Stack Memory



Heap Space



Garbage collector

В куче работает [Garbage collector](#) — программа, работающая на виртуальной машине Java, которая избавляется от объектов, к которым невозможно получить доступ.

Разные JVM могут иметь различные алгоритмы сборки мусора, также существуют разные сборщики мусора.

Типы сборщиков мусора: Serial, Parallel, G1, Z1, Concurrent Mark Sweep (CMS) GC, Epsilon, Shenandoah, ZGC

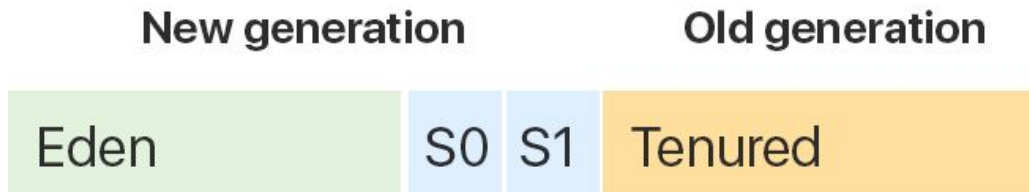
Serial GC. Процесс работы.

Сборку мусора мы запрашиваем при помощи `System.gc()`. Но эта команда не обязует JVM запустить процесс сборки мусора.

Как уже было упомянуто выше, куча разбита на 2 области: New generation и Old generation.

New generation (младшее поколение) включает в себя 3 региона: Eden, Survivor 0 и Survivor 1.

Old generation включает в себя регион Tenured.



Serial GC. Процесс работы.

Что происходит, когда мы создаем в Java объект? - объект попадает в Eden. Если мы создали уже много объектов и в Eden уже нет места, срабатывает сборщик мусора и освобождает память. Это, так называемая, малая сборка мусора — на первом проходе он очищает область Eden и кладёт “выжившие” объекты в регион Survivor 0. Таким образом регион Eden полностью высвобождается.

Если произошло так, что область Eden снова была заполнена, garbage collector начинает работу с областью Eden и областью Survivor 0, которая занята на данный момент. После очищения выжившие объекты попадут в другой регион — Survivor 1, а два остальных останутся чистыми. При последующей сборке мусора в качестве региона назначения опять будет выбран Survivor 0. Именно поэтому важно, чтобы один из регионов Survivor всегда был пустым.

Serial GC. Процесс работы.

JVM следит за объектами, которые постоянно копируются и перемещаются из одного региона в другой. И для того, чтобы оптимизировать данный механизм, после определённого порога сборщик мусора перемещает такие объекты в регион Tenured.

Когда в Tenured места для новых объектов не хватает, происходит полная сборка мусора — Mark-Sweep-Compact.

Во время этого механизма определяется, какие объекты больше не используются, регион очищается от этих объектов, и область памяти Tenured дефрагментируется, т.е. последовательно заполняется нужными объектами.

Mark-Sweep-Compact

Before



After



Источники

1. <https://habr.com/ru/articles/471772/>
2. <https://www.baeldung.com/java-classloaders>
3. <https://sky.pro/media/ponimanie-oshibki-stackoverflowerror-v-java/>
4. <https://sky.pro/media/parametry-xms-i-xmx-pri-zapuske-jvm/>
5. <https://vc.ru/dev/397062-sborka-musora-v-java-cto-eto-takoe-i-kak-rabotaet-v-jvm>
6. <https://habr.com/ru/companies/otus/articles/553996/>